

Newton-Driven Deep Unrolling Networks: From CS to LRR

Xianchao Xiu

Department of Automation



CSIAM Big Data and Artificial Intelligence, July 17-19, 2026

Joint work with [Changhua He](#) (SHU), [Xinjie Wang](#) (SUEP) and others

Outline

Introduction

Compressed Sensing

Low-Rank Representation

Conclusions

Why

- ▶ Iterative shrinkage thresholding algorithm (ISTA)

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$$

- ▶ Step 1: Compute gradient

$$r^{k+1} = x^k - \rho A^\top (Ax^k - b)$$

- ▶ Step 2: Compute soft-thresholding

$$\begin{aligned} x^{k+1} &= \arg \min_x \frac{1}{2} \|x - r^{k+1}\|^2 + \lambda \|x\|_1 \\ &= \text{Prox}_{\lambda \|\cdot\|_1}(x^k - \rho A^\top (Ax^k - b)) \end{aligned}$$

- ▶ Some issues need to be addressed
 - ▶ How to develop optimization algorithms?
 - ▶ How to construct regularization terms?
 - ▶ How to realize automated design?

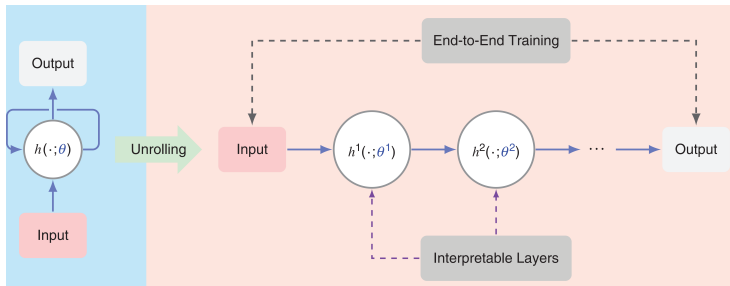
LISTA

- Gregor-LeCun, Learning Fast Approximations of Sparse Coding, [ICML](#), 2010

$$x^{k+1} = \text{Prox}_{\lambda\|\cdot\|_1}(x^k - \rho A^\top (Ax^k - b))$$

$\Downarrow$

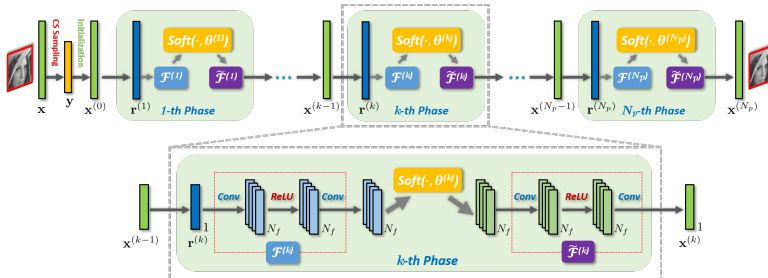
$$x^{k+1} = h_{\theta^k}(x^k - \rho A^\top (Ax^k - b))$$



- Chen-Liu-Wang et al, Theoretical Linear Convergence of Unfolded ISTA and Its Practical Weights and Thresholds, [NeurIPS](#), 2018

ISTA-Net

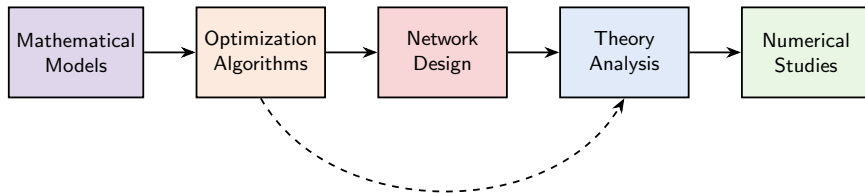
- Zhang-Ghanem, ISTA-Net: Interpretable Optimization-Inspired Deep Network for Image Compressive Sensing, [CVPR](#), 2018



Algorithm	CS Ratio							Time CPU/GPU
	50%	40%	30%	25%	10%	4%	1%	
TVAL3 [22]	33.55	31.46	29.23	27.92	22.99	18.75	16.43	3.135s/—
D-AMP [28]	35.92	33.56	30.39	28.46	22.64	18.40	5.21	51.21s/—
IRCNN [49]	36.23	34.06	31.18	30.07	24.02	17.56	7.70	—/68.42s
SDA [30]	28.95	27.79	26.63	25.34	22.65	20.12	17.29	—/0.0032s
ReconNet [21]	31.50	30.58	28.74	25.60	24.28	20.63	17.27	—/0.016s
ISTA-Net	<u>37.43</u>	<u>35.36</u>	<u>32.91</u>	<u>31.53</u>	25.80	<u>21.23</u>	<u>17.30</u>	0.923s/0.039s
ISTA-Net ⁺	38.07	36.06	33.82	32.57	26.64	21.31	17.34	1.375s/0.047s

Motivation

- ▶ Yang-Sun-Li et al, ADMM-CSNet: A Deep Learning Approach for Image Compressive Sensing, [IEEE TPAMI](#), 2020
- ▶ Song-Zhang-Zhou et al, SOR-Unrolled Learning-to-Solve Method for Absolute Value Equations, [JORSC](#), 2026
- ▶ Oshin-Ghosh-Saravanos et al, Deep FlexQP: Accelerated Nonlinear Programming via Deep Unfolding, [ICLR](#), 2026
- ▶ Saravanos-So-Ahmad et al, Learning-to-Optimize via Deep Unfolded Flows, [ICML](#), 2026
- ▶ Deng-Zhang-Jiang et al, DeepSN-Net: Deep Semi-Smooth Newton Driven Network for Blind Image Restoration, [IEEE TPAMI](#), 2025



Outline

Introduction

Compressed Sensing

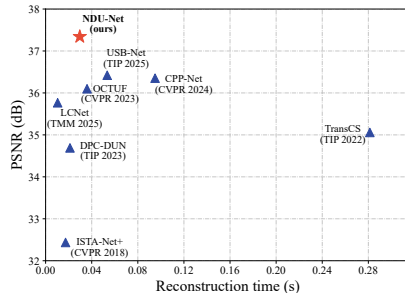
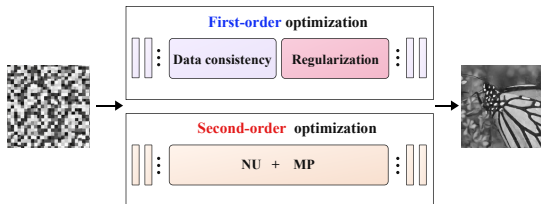
Low-Rank Representation

Conclusions

- Recall the CS optimization problem

$$\min_{x \in \mathbb{R}^n} \underbrace{\frac{1}{2} \|Ax - b\|^2}_{f(x)} + \lambda \mathcal{R}(x)$$

- Iterative optimization: ISTA, FISTA, AMP, PDHG, ADMM, **SSNAL**, **NHTP**
- Deep unrolling: LISTA, ALISTA, ISTA-Net, ISTA-Net+, LDAMP, ADMM-CSNet



Step 1

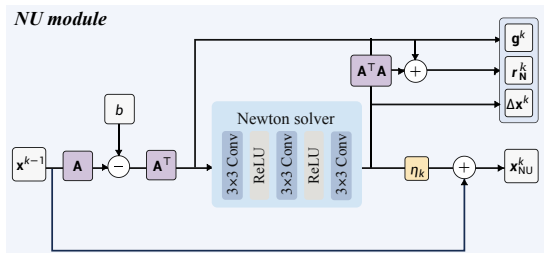
- (ISTA) Compute gradient

$$r^{k+1} = x^k - \rho \nabla f(x^k)$$

- (NDU-Net) The Newton update (NU) satisfies

$$\begin{cases} \nabla^2 f(x^k) \Delta x^k + \nabla f(x^k) = 0 \\ x_{\text{NU}}^k \leftarrow x^k + \eta_k \Delta x^k \end{cases}$$

Define the Newton residual as $r_N^k = A^\top A \Delta x^k + \nabla f(x^k)$



Step 2

- (ISTA) Compute soft-thresholding

$$x^{k+1} = \text{Prox}_{\lambda \|\cdot\|_1}(x^k - \rho \nabla f(x^k))$$

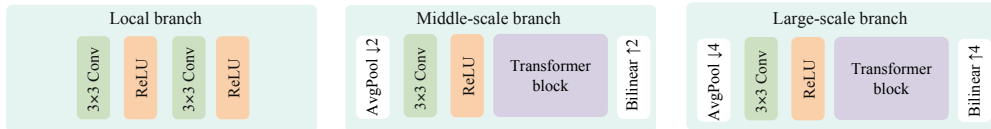
- (NDU-Net) Extract multi-scale prior (MP) information

$$y_{MS}^k = \mathcal{P}_{MS}^k(x_{NU}^k)$$

Encode Newton guidance information

$$y_N^k = \mathcal{E}_N^k(g^k, r_N^k, \Delta x^k)$$

$$\Rightarrow y_{\text{guided}}^k = y_{MS}^k + y_N^k$$



Step 2

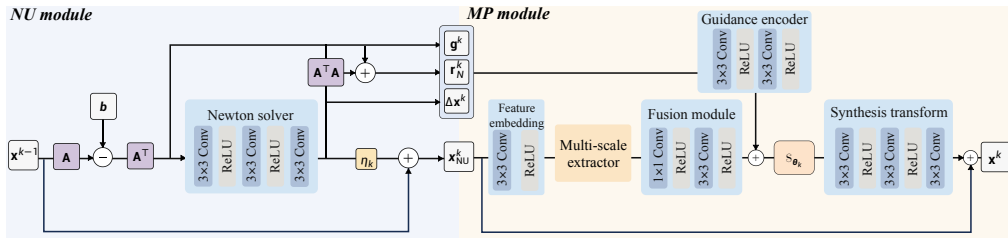
- (NDU-Net) Apply a learnable soft-thresholding operation

$$y_{\text{shrink}}^k = \mathcal{S}_{\theta_k}(y_{\text{guided}}^k)$$

$$\mathcal{S}_{\theta_k}(z) = \text{sign}(z) \odot \max(|z| - \theta_k, 0)$$

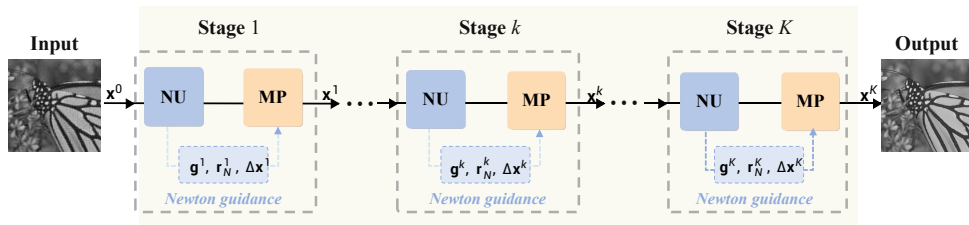
Transform back to the image domain

$$r_{\text{prior}}^k = \mathcal{H}^k(y_{\text{shrink}}^k)$$



NDU-Net

► Overall architecture



► In the k -th stage

$$x^{k+1} = x_{\text{NU}}^k \text{ (Step 1)} + r_{\text{prior}}^k \text{ (Step 2)}$$

Training loss

$$\begin{aligned} \ell_{\text{total}} &= \ell_{\text{rec}} + \sigma \ell_{\text{Newton}} \\ &= \frac{1}{N} \|x^K - x\|_1 + \sigma \left(\frac{1}{K} \sum_{k=0}^{K-1} \left(\frac{1}{N} \|r_N^k\|_1 \right) \right) \end{aligned}$$

Experiments

- ▶ Compared methods
 - ▶ ISTA-Net+: Zhang-Ghanem, [CVPR](#), 2018
 - ▶ TransCS: Shen-Gan-Ning et al, [IEEE TIP](#), 2022
 - ▶ DPC-DUN: Song-Chen-Zhang, [IEEE TIP](#), 2023
 - ▶ OCTUF: Song-Mou-Wang et al, [CVPR](#), 2023
 - ▶ CPP-Net: Guo-Gan, [CVPR](#), 2024
 - ▶ LCNet: Yang-Fernando-Qin, [IEEE TMM](#), 2025
 - ▶ USB-Net: Guo-Gan, [IEEE TIP](#), 2025
- ▶ Implementation details
 - ▶ Hyperparameters: $\sigma = 0.01$, $K = 6$
 - ▶ Learnable parameters: $\mathcal{R}(x)$, λ
 - ▶ CS ratios: 0.10, 0.25, 0.40, 0.50
 - ▶ Metrics: PSNR/SSIM ($\uparrow$)
- ▶ <https://github.com/xianchaoxiu/NDU-Net>

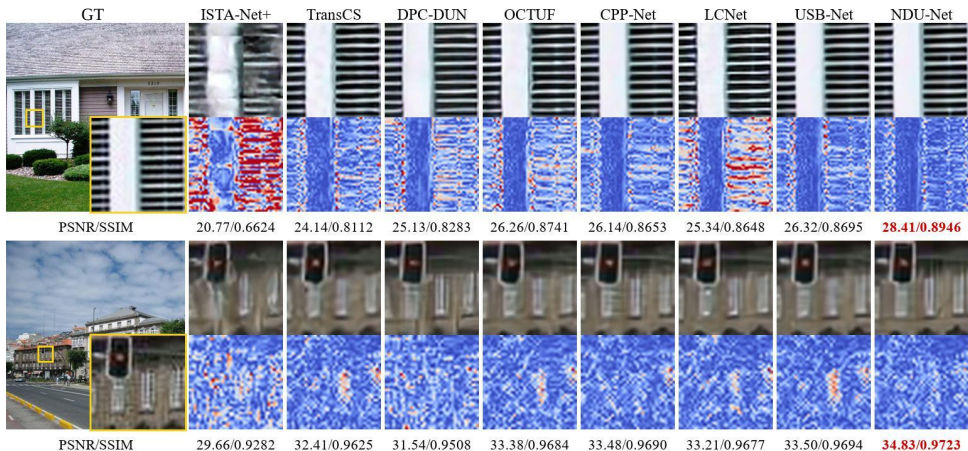
Results

- More reconstruction comparisons are provided

Datsets	Methods	CS ratios				Average
		0.10	0.25	0.40	0.50	
LIVE29	ISTA-Net+ (CVPR'18)	25.05/0.7104	29.42/0.8648	32.55/0.9275	34.55/0.9514	30.39/0.8635
	TransCS (TIP'22)	27.66/0.8196	32.05/0.9251	35.51/0.9633	37.74/0.9767	33.24/0.9212
	DPC-DUN (TIP'23)	26.83/0.7812	31.17/0.9008	34.43/0.9477	36.43/0.9649	32.22/0.8987
	OCTUF (CVPR'23)	28.23/0.8345	32.69/0.9327	36.19/0.9671	38.52/0.9795	33.91/0.9285
	CPP-Net (CVPR'24)	28.46/0.8414	32.82/0.9343	36.20/0.9672	38.37/0.9790	33.96/0.9305
	LCNet (TMM'25)	28.19/0.8349	32.64/0.9325	36.08/0.9667	38.34/0.9791	33.81/0.9283
	USB-Net (TIP'25)	28.48/0.8420	32.87/0.9348	36.29/0.9676	38.46/0.9793	34.03/0.9309
	NDU-Net	29.62/0.8517	34.07/0.9399	37.55/0.9708	39.62/0.9812	35.22/0.9359
OST300	ISTA-Net+ (CVPR'18)	24.78/0.6896	28.53/0.8433	31.34/0.9116	33.20/0.9396	29.46/0.8460
	TransCS (TIP'22)	27.31/0.8019	31.07/0.9096	34.17/0.9534	36.25/0.9701	32.20/0.9088
	DPC-DUN (TIP'23)	26.25/0.7561	29.93/0.8792	32.81/0.9335	34.69/0.9554	30.92/0.8811
	OCTUF (CVPR'23)	27.77/0.8148	31.60/0.9175	34.61/0.9572	36.69/0.9726	32.67/0.9155
	CPP-Net (CVPR'24)	27.93/0.8207	31.67/0.9185	34.66/0.9573	36.68/0.9724	32.74/0.9172
	LCNet (TMM'25)	27.73/0.8151	31.53/0.9169	34.53/0.9568	36.60/0.9723	32.60/0.9153
	USB-Net (TIP'25)	27.95/0.8214	31.71/0.9191	34.71/0.9578	36.71/0.9726	32.77/0.9177
	NDU-Net	29.22/0.8314	33.00/0.9239	36.01/0.9604	37.94/0.9739	34.04/0.9224

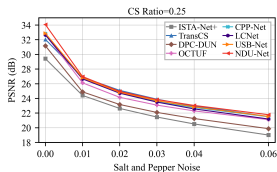
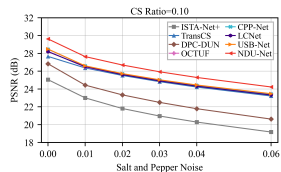
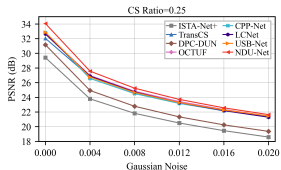
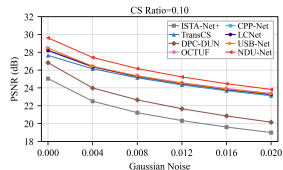
Visualization

► Visual comparisons on OST300

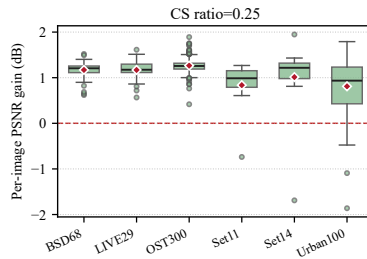


Robustness

► Noise corruption



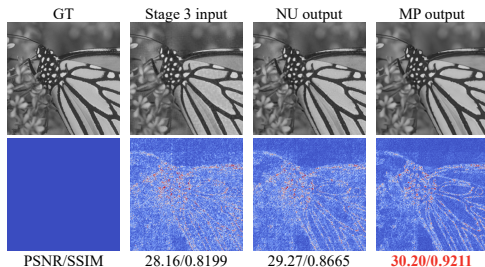
► Per-image PSNR gain



Ablations

► Effects of NU and MP

Variants	LIVE29	Set11	BSD68
w/o NU	33.91/0.9307	36.92/0.9625	36.80/0.9505
w/o MP	33.16/0.9257	35.70/0.9545	36.21/0.9483
Full	34.43/0.9409	37.34/0.9657	37.61/0.9599

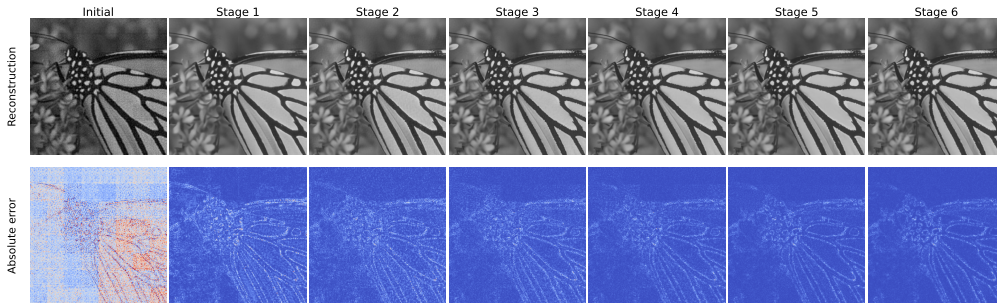


► Effects of stages

Stages	PSNR	SSIM	Params
1	33.44	0.9351	1.001
2	35.73	0.9550	1.999
3	36.67	0.9612	2.996
4	37.02	0.9635	3.994
5	37.29	0.9653	4.991
6	37.34	0.9657	5.989
7	37.31	0.9654	6.987

Remarks

- ▶ How to **automatically** determine the number of stages?



- ▶ Does it exhibit quadratic convergence under some conditions?
- ▶ What is the connection between learned solvers and true Newton directions?

Outline

Introduction

Compressed Sensing

Low-Rank Representation

Conclusions

LRR

- Liu-Lin-Yan et al, Robust Recovery of Subspace Structures by Low-Rank Representation, *IEEE TPAMI*, 2013

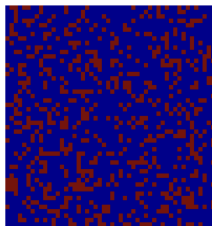
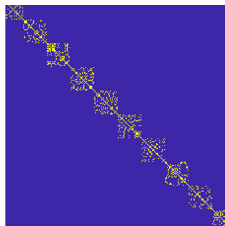
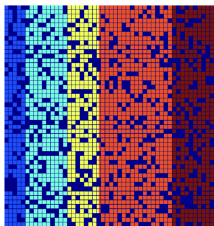
$$\min_{Z, E} \text{rank}(Z) + \lambda \|E\|_0$$

$$\text{s.t. } X = XZ + E$$

$\Rightarrow$

$$\min_{Z, E} \|Z\|_* + \lambda \|E\|_1$$

$$\text{s.t. } X = XZ + E$$



- Broad applications: clustering, classification, denoising, detection, fusion
- Related to low-rank regularization, low-rank approximation, low-rank decomposition

Model

- How to construct regularization terms?

$$\begin{array}{ll} \min_{Z, E} & \text{rank}(Z) + \lambda \|E\|_0 \\ \text{s.t.} & X = XZ + E \end{array} \quad \Rightarrow \quad \begin{array}{ll} \min_{Z, E} & \mathcal{R}(Z) + \lambda \mathcal{S}(E) \\ \text{s.t.} & X = XZ + E \end{array}$$

- Augmented Lagrangian method (ALM)

$$\begin{array}{ll} \min_{J, Z, E} & \mathcal{R}(J) + \lambda \mathcal{S}(E) \\ \text{s.t.} & X = XZ + E, \quad Z = J \end{array}$$

which derives

$$\begin{aligned} \mathcal{L}_\beta(J, E, Z, Y_1, Y_2) &= \mathcal{R}(J) + \lambda \mathcal{S}(E) + \langle Y_1, Z - J \rangle \\ &+ \frac{\beta}{2} \|Z - J\|_F^2 + \langle Y_2, X - XZ - E \rangle + \frac{\beta}{2} \|X - XZ - E\|_F^2 \end{aligned}$$

Algorithm

- Update variables according to

$$\begin{cases} (J, E, Z) \leftarrow \arg \min_{J, E, Z} \mathcal{L}_\beta(J, E, Z, Y_1, Y_2) \\ Y_1 \leftarrow Y_1 + \beta(Z - J) \\ Y_2 \leftarrow Y_2 + \beta(X - XZ - E) \end{cases}$$

- First-order optimality conditions

$$\begin{cases} 0 \in \partial \mathcal{R}(J) - Y_1 + \beta(J - Z) \\ 0 \in \lambda \partial \mathcal{S}(E) - Y_2 - \beta(X - XZ - E) \\ 0 = Y_1 + \beta(Z - J) - X^\top Y_2 - \beta X^\top (X - XZ - E) \end{cases}$$

which derives

$$J = \text{Prox}_{1/\beta, \mathcal{R}}(Z + Y_1/\beta), \quad E = \text{Prox}_{\lambda/\beta, \mathcal{S}}(X - XZ + Y_2/\beta)$$

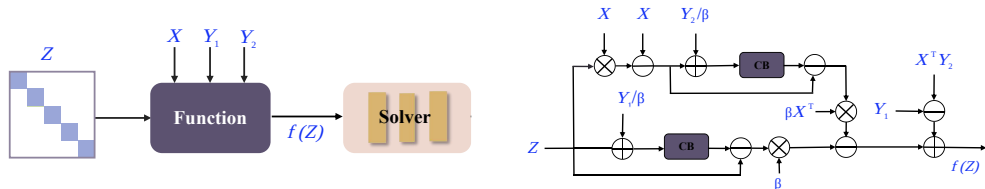
- Denote the nonlinear equation

$$f(Z) = Y_1 + \beta (Z - \text{Prox}_{1/\beta, \mathcal{R}}(Z + Y_1/\beta)) - X^\top Y_2 \\ - \beta X^\top (X - XZ - \text{Prox}_{\lambda/\beta, \mathcal{S}}(X - XZ + Y_2/\beta))$$

$\Downarrow$

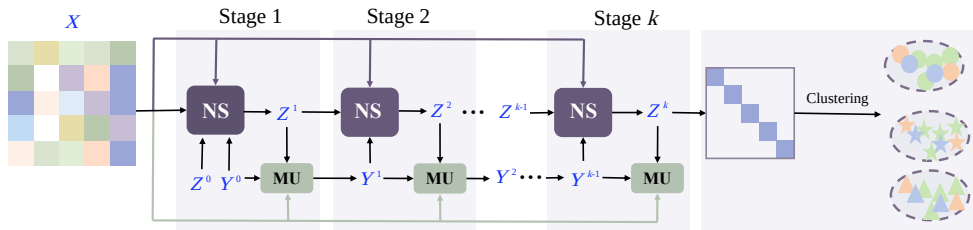
$$\begin{cases} g(Z)\Delta Z = -f(Z) \\ Z \leftarrow Z + \eta\Delta Z \end{cases}$$

- Detailed structure: NS (Newton solver), CB (convolutional block)



LRRSSN-Net

► Overall architecture



► Training loss

$$\ell_{\text{total}} = \ell_{\text{self}} + \sigma_1 \ell_{\text{fun}} + \sigma_2 \ell_{\text{reg}}$$

$$= \left(\|X - XZ^K - E^K\|_F^2 + \gamma \|E^K\|_1 \right) + \sigma_1 \left(\frac{1}{N} \sum_{l=1}^N \|f(Z^K)\|_1 \right) + \sigma_2 \text{tr}(L_{Z^K})$$

Experiments

- ▶ Compared methods
 - ▶ IDR: Wei-Liu-Zhou et al, [IEEE TKDE](#), 2023
 - ▶ TS1-LLRR: Hu-Qu-Liu et al, [KBS](#), 2024
 - ▶ JSSC: Zhu-Xiu-Liu et al, [ESWA](#), 2025
 - ▶ ISC: Zhang-Zhou-Yao et al, [IEEE TPAMI](#), 2026
 - ▶ DSC-Net: Ji-Zhang-Li et al, [NeurIPS](#), 2017
 - ▶ PRO-DSC: Meng-Huang-He et al, [ICLR](#), 2025
 - ▶ DCSSC: Oliveira-Queiroz-Zanchettin et al, [Neuro](#), 2025
- ▶ Implementation details
 - ▶ Hyperparameters: $\sigma_1 = 0.01$, $\sigma_2 = 1$, $K=6$
 - ▶ Learnable parameters: $\mathcal{R}(J)$, $\mathcal{S}(E)$, λ , β
 - ▶ Clustering: k -means
 - ▶ Metrics: ACC ($\uparrow$)
- ▶ <https://github.com/xianchaoxiu/LRRSSN-Net>

Results

► Clustering comparisons

Methods	Datasets								Average	
	AR	FRDUE	MNIST	USPS	GLOMA	ISOLET	MSTAR	TDT2	ACC	Time
IDR (TKDE'23)	72.64%	95.31%	58.61%	79.80%	66.00%	67.65%	84.50%	84.10%	76.08%	225.80
TS1-LLRR (KBS'24)	76.46%	88.64%	49.65%	79.48%	61.60%	65.64%	87.61%	84.51%	74.20%	265.10
JSSC (ESWA'25)	78.09%	91.31%	46.78%	69.22%	50.00%	66.61%	65.99%	59.37%	65.92%	358.60
ISC (TPAMI'26)	54.42%	88.19%	44.02%	65.30%	60.00%	81.02%	43.13%	51.75%	60.98%	118.50
DSC-Net (NeurIPS'17)	45.19%	82.53%	66.07%	90.00%	70.00%	41.41%	82.53%	12.45%	61.27%	23.23
PRO-DSC (ICLR'25)	63.75%	96.09%	45.39%	67.60%	59.40%	52.00%	66.41%	88.08%	67.34%	0.03
DCSSC (Neuro'25)	83.68%	80.46%	87.25%	90.30%	84.00%	86.00%	91.85%	78.35%	85.24%	0.85
LRRSSN-Net	89.95%	96.02%	93.43%	99.40%	98.00%	97.72%	99.72%	96.40%	96.33%	0.21



(a) IDR



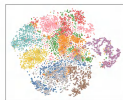
(b) TS1-LLRR



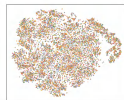
(c) JSSC



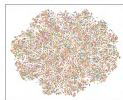
(d) ISC



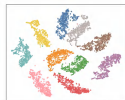
(e) DSC-Net



(f) PRO-DSC



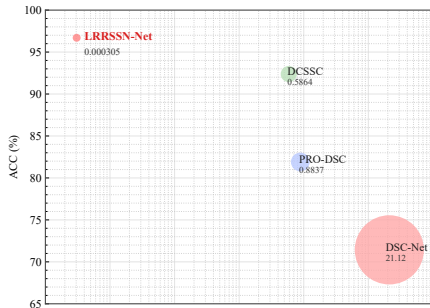
(g) DCSSC



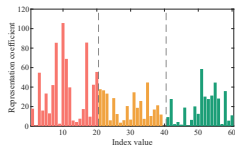
(h) LRRSSN-Net

Analysis

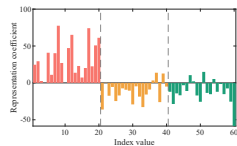
► Parameter efficiency



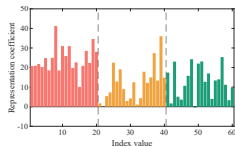
► Representation efficiency



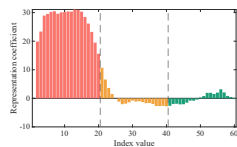
(a) TS1-LLRR



(b) JSSC



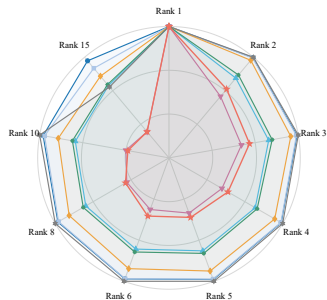
(c) DSC-Net



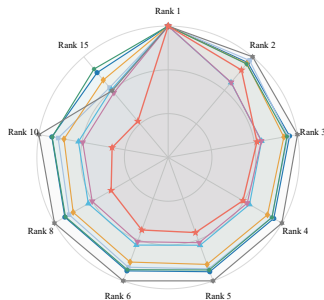
(d) LRRSSN-Net

Low-Rankness

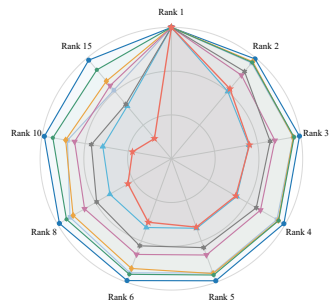
► Normalized singular-value spectrum



(a) USPS



(b) ISOLET



(c) MSTAR

— IDR — TS1LLRR — JSSC — ISC — DSC-Net — PRO-DSC — DGSSC — LRRSSN-Net

Remarks

- ▶ Why do some datasets perform well while others do not?
- ▶ How to avoid k -means in clustering, i.e., **end-to-end clustering**?
- ▶ **What are the advantages of large language models for clustering?**

Large Language Models Enable Few-Shot Clustering

**Vijay Viswanathan¹, Kiril Gashteovski^{2,3},
Carolyn Lawrence², Tongshuang Wu¹, Graham Neubig¹**

¹Carnegie Mellon University, USA, ²NEC Laboratories Europe, Germany

³Center for Advanced Interdisciplinary Research, Ss. Cyril and Methodius Uni. of Skopje, Germany

Abstract

Unlike traditional unsupervised clustering, semi-supervised clustering allows users to provide meaningful structure to the data, which helps the clustering algorithm to match the user's intent. Existing approaches to semi-

2004). These interfaces have all been shown to give experts control of the final clusters. However, for large, real-world datasets with a large number of possible clusters, the feedback cost required by interactive clustering algorithms can be immense.

Outline

Introduction

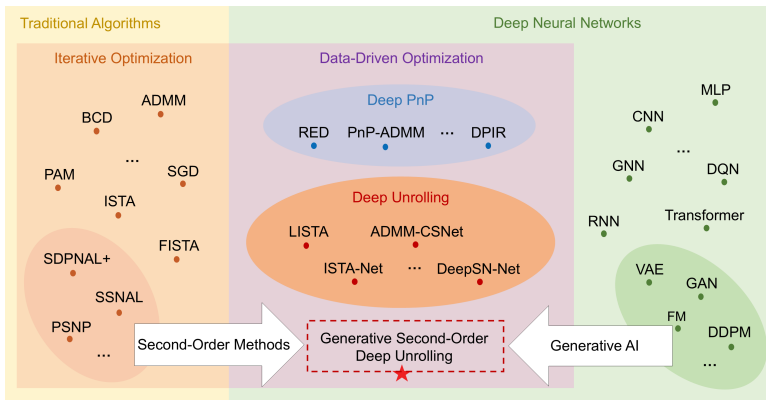
Compressed Sensing

Low-Rank Representation

Conclusions

Conclusions

- ▶ How to develop optimization algorithms? $\Rightarrow$ Second-order methods
- ▶ How to construct regularization terms? $\Rightarrow$ Deep unrolling
- ▶ How to realize automated design? $\Rightarrow$ Generative AI



References

- ▶ He-Xiu, Newton Deep Unfolding for Compressed Sensing, [submitted to AAAI](#), 2026
- ▶ Chen-Wang-Xiu et al, Rethinking Low-Rank Representation with Deep Semi-Smooth Newton, [PRCV](#), 2026
- ▶ Liu-Jin-Xiu et al, STAR-Net: An Interpretable Model-Aided Network for Remote Sensing Image Denoising, [PR](#), 2026
- ▶ Liu-Han-Xiu et al, Lightweight Deep Unfolding Networks with Enhanced Robustness for Infrared Small Target Detection, [IEEE TIP](#), 2026

Thank you for your attention!

xcxiu@shu.edu.cn